Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки и словари менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

## Задача 1. Заменить цифру

**2 балла**

`old` и `new` - цифры от 0 до 9. Замените каждое вхождение цифры `old` на цифру `new`. Остальные цифры оставьте без изменения.

`number` - целое неотрицательное число. Порядок разрядов сохраняется. Верните целое число; начальные нули в результате не сохраняются. Число `0` считайте записью из одной цифры `0`. Преобразовывать число или отдельные цифры в строки нельзя.

```
def replace_digit(number: int, old: int, new: int) -> int:
    ...
```

Примеры:

```
assert replace_digit(120203, 2, 7) == 170703
assert replace_digit(1200, 1, 3) == 3200
assert replace_digit(505, 5, 0) == 0
assert replace_digit(1002, 0, 3) == 1332
assert replace_digit(123, 8, 4) == 123
assert replace_digit(0, 0, 7) == 7
```

## Задача 2. Сокращение слов: сохранить начало

**2 балла**

`text` - строка из букв и пробелов, `limit` - целое положительное число.

Каждое слово, длина которого строго больше `limit`, замените на его первые `limit` букв, добавив после них три точки `...`. Слова длины не больше `limit` оставьте без изменения. Точки не входят в ограничение `limit`.

Верните одну строку из полученных слов в исходном порядке, с одним пробелом между ними и без пробелов по краям. Сохраните регистр букв и все повторы слов. В исходной строке пробелов между словами, в начале и конце может быть несколько. Если слов нет, верните пустую строку.

```
def shorten_word_starts(text: str, limit: int) -> str:
    ...
```

Примеры:

```
assert shorten_word_starts('Мы изучаем Python', 3) == 'Мы изу... Pyt...'
assert shorten_word_starts('  Кит кот  Кит ', 3) == 'Кит кот Кит'
assert shorten_word_starts('домик дом', 3) == 'дом... дом'
assert shorten_word_starts('Я', 1) == 'Я'
assert shorten_word_starts('   ', 2) == ''
assert shorten_word_starts('', 2) == ''
```

## Задача 3. Группы между нулями

**2,5 балла**

`values` - список целых чисел. Разделителями служат нули; все ненулевые числа входят в группы.

Верните список непустых групп. Каждая группа - новый список из элементов между разделителями. Разделители в результат не входят. Порядок групп и элементов сохраняется. Несколько разделителей подряд, в начале или в конце списка не создают пустых групп. Если групп нет, верните пустой список.

```
def split_at_zeros(values: list[int]) -> list[list[int]]:
    ...
```

Примеры:

```
assert split_at_zeros([0, 2, -1, 0, 0, 3, 4, 0]) == [[2, -1], [3, 4]]
assert split_at_zeros([1, -2, 3]) == [[1, -2, 3]]
assert split_at_zeros([1, 0, 1]) == [[1], [1]]
assert split_at_zeros([0, 0]) == []
assert split_at_zeros([]) == []
```

## Задача 4. Выдача книг

**3,5 балла**

`stock` - список названий доступных книг. Каждая запись соответствует одному экземпляру: если название встречается три раза, доступны три экземпляра этой книги. `requests` - список названий книг в порядке запросов на выдачу. Каждый запрос - на один экземпляр.

Запросы обрабатываются по порядку. Для каждого запроса верните `True`, если его удалось выполнить, и `False` в противном случае. Успешный запрос расходует один экземпляр. Если доступных экземпляров указанной книги уже нет, нужно отказать. Отказ ничего не меняет.

Исходные списки менять нельзя. Каждый вызов функции независим от предыдущих. Строки сравниваются точно, с учётом регистра.

Среднее время работы должно быть `O(n + m)`, включая предварительную обработку первого списка, где `n` - длина первого списка, `m` - число запросов. Считайте, что проверка наличия строкового ключа, получение и изменение его значения, добавление записи в словарь в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def lend_books(stock: list[str], requests: list[str]) -> list[bool]:
    ...
```

Примеры:

```
assert lend_books(['Дюна', 'Дюна', 'Солярис'],
                  ['Дюна', 'Солярис', 'Дюна', 'Дюна', 'Мир']) == [
    True, True, True, False, False,
]
assert lend_books([], ['Дюна']) == [False]
assert lend_books(['Дюна'], []) == []
stock = ['Дюна']
assert lend_books(stock, ['Дюна', 'Дюна']) == [True, False]
assert lend_books(stock, ['Дюна']) == [True]
assert stock == ['Дюна']
```

Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки и словари менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

## Задача 1. Поменять две цифры местами

**2 балла**

`first` и `second` - цифры от 0 до 9. В исходном числе замените каждую цифру `first` на `second`, а каждую цифру `second` на `first`. Замены выполняются одновременно: уже заменённую цифру повторно не меняйте. Остальные цифры остаются прежними. Если `first == second`, число не меняется.

`number` - целое неотрицательное число. Порядок разрядов сохраняется. Верните целое число; начальные нули в результате не сохраняются. Число `0` считайте записью из одной цифры `0`. Преобразовывать число или отдельные цифры в строки нельзя.

```
def swap_digits(number: int, first: int, second: int) -> int:
    ...
```

Примеры:

```
assert swap_digits(120203, 2, 3) == 130302
assert swap_digits(1200, 1, 2) == 2100
assert swap_digits(1002, 1, 0) == 112
assert swap_digits(505, 5, 0) == 50
assert swap_digits(123, 2, 2) == 123
assert swap_digits(0, 0, 7) == 7
```

## Задача 2. Сокращение слов: сохранить конец

**2 балла**

`text` - строка из букв и пробелов, `limit` - целое положительное число.

Каждое слово, длина которого строго больше `limit`, замените на его последние `limit` букв, добавив перед ними три точки `...`. Слова длины не больше `limit` оставьте без изменения. Точки не входят в ограничение `limit`.

Верните одну строку из полученных слов в исходном порядке, с одним пробелом между ними и без пробелов по краям. Сохраните регистр букв и все повторы слов. В исходной строке пробелов между словами, в начале и конце может быть несколько. Если слов нет, верните пустую строку.

```
def shorten_word_ends(text: str, limit: int) -> str:
    ...
```

Примеры:

```
assert shorten_word_ends('Мы изучаем Python', 3) == 'Мы ...аем ...hon'
assert shorten_word_ends('  Кит кот  Кит ', 3) == 'Кит кот Кит'
assert shorten_word_ends('домик дом', 3) == '...мик дом'
assert shorten_word_ends('Я', 1) == 'Я'
assert shorten_word_ends('   ', 2) == ''
assert shorten_word_ends('', 2) == ''
```

## Задача 3. Группы между отрицательными числами

**2,5 балла**

`values` - список целых чисел. Разделителями служат все отрицательные числа. Нули и положительные числа входят в группы.

Верните список непустых групп. Каждая группа - новый список из элементов между разделителями. Разделители в результат не входят. Порядок групп и элементов сохраняется. Несколько разделителей подряд, в начале или в конце списка не создают пустых групп. Если групп нет, верните пустой список.

```
def split_at_negatives(values: list[int]) -> list[list[int]]:
    ...
```

Примеры:

```
assert split_at_negatives([-1, 2, 0, -3, -4, 3, 4, -2]) == [[2, 0], [3, 4]]
assert split_at_negatives([1, 0, 3]) == [[1, 0, 3]]
assert split_at_negatives([1, -1, 1]) == [[1], [1]]
assert split_at_negatives([-1, -2]) == []
assert split_at_negatives([]) == []
```

## Задача 4. Запись на занятия

**3,5 балла**

`places` - список названий занятий, на которые есть свободные места. Каждая запись соответствует одному месту: если название встречается три раза, на это занятие есть три места. `requests` - список названий занятий в порядке заявок. Каждая заявка - на одно место.

Запросы обрабатываются по порядку. Для каждого запроса верните `True`, если его удалось выполнить, и `False` в противном случае. Успешный запрос расходует одно место. Если свободных мест на указанное занятие уже нет, нужно отказать. Отказ ничего не меняет.

Исходные списки менять нельзя. Каждый вызов функции независим от предыдущих. Строки сравниваются точно, с учётом регистра.

Среднее время работы должно быть `O(n + m)`, включая предварительную обработку первого списка, где `n` - длина первого списка, `m` - число запросов. Считайте, что проверка наличия строкового ключа, получение и изменение его значения, добавление записи в словарь в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def book_sessions(places: list[str], requests: list[str]) -> list[bool]:
    ...
```

Примеры:

```
assert book_sessions(['Python', 'Python', 'SQL'],
                     ['Python', 'SQL', 'Python', 'Python', 'Excel']) == [
    True, True, True, False, False,
]
assert book_sessions([], ['Python']) == [False]
assert book_sessions(['Python'], []) == []
places = ['Python']
assert book_sessions(places, ['Python', 'Python']) == [True, False]
assert book_sessions(places, ['Python']) == [True]
assert places == ['Python']
```